ZIDIO DEVELOPMENT · DATA SCIENCE & ANALYTICS

INDUSTRY EDITION · V2.0

RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Predictive Demand • Customer Segmentation • Churn Analysis • Inventory Optimization

Janhavi Prakash Amberkar
Data Science & Analytics Intern · Mentor: Chandan Mishra
November 2026
Dataset: UCI Online Retail II · 1.07 M transactions · Python · Prophet · PyTorch · XGBoost · Streamlit · MLflow

Project links

Source code (GitHub)	https://github.com/Janhavi-Amberkar/retailpulse
Live demo (public HTTPS)	https://retailpulse-77wtccmydlnjaosuqdorjv.streamlit.app
Demo video & feedback video	https://drive.google.com/drive/folders/1CktH6fhtpEjAqZHLpW4jymXMZ-zzh_l4
This report (PDF, Drive)	https://drive.google.com/drive/folders/1CktH6fhtpEjAqZHLpW4jymXMZ-zzh_l4

1 · Project Overview

Vision & objectives

Retailers lose revenue to stock-outs on one side and tied-up capital on the other, and they rarely know which customers are about to leave. RetailPulse is an end-to-end data-science platform that ingests **1,067,371** real transaction lines, cleans them through an automated ETL, and delivers four decisions: **how much will sell** (30-day demand forecast), **who our customers are** (RFM + behavioural segments), **who is leaving** (churn risk with explanations) and **what to reorder** (safety stock, reorder points, EOQ). Everything is exposed in an interactive Streamlit dashboard and a JWT-secured API, and operated with MLOps practices (MLflow, Evidently AI drift detection, retraining loop, Docker/K8s/CI).

Target users & use cases

- **Merchandising / supply-chain planners** – weekly purchase plan, what-if on lead time, service level and demand shocks.
- **CRM / marketing** – at-risk customer lists ranked by expected revenue loss, win-back ROI simulator.
- **Management** – KPI overview, spec scoreboard, exportable executive PDF.
- **Data / ML engineers** – experiment tracking, drift monitor, retraining policy, audit trail.

Business value delivered (measured on real data)

11.9%
weekly forecast MAPE (7-fold CV)

5.3%
error on 30-day total (hold-out)

0.89
churn precision in top-20% risk

87.4%
fill rate vs 70.0% naive

- Forecast-driven replenishment cut back-tested **total cost by 56%** vs a moving-average policy with no buffer and **11%** vs the stronger moving-average + safety-stock baseline (details in §6).
- The *Critical* churn tier holds **520** customers worth ≈ **£370,482** of expected 180-day revenue; targeting the top-20% by risk is 1.57× more efficient than random.
- Customer base concentration: the top segment (“Champions”, 1,160 customers) generates **73%** of revenue.

Non-functional goals vs measured

Spec target	Measured on	Result	Status
Forecast MAPE ≤ 12 %	weekly MAPE (CV) / hold-out	11.9 % / 11.2 %	✔ met
	daily MAPE (CV) / hold-out	30.9 % / 19.5 %	✘ not met
Churn AUC-ROC ≥ 0.88	out-of-time test	0.806	✘ not met
Churn precision@top20 % ≥ 0.75	out-of-time test	0.889	✔ met
6–8 customer segments	K-Means, silhouette-selected	6 segments	✔ met
Daily batch < 5 min	full ETL transform + aggregation	7.3 s for 3.2 M rows	✔ met
10 M+ transactions / month	linear extrapolation of the above	≈ 23 s for 10 M rows	✔ met

Spec target	Measured on	Result	Status
Observability	MLflow runs · Evidently drift · Prometheus	implemented	✔ met

Honest reading of the misses. **Daily** MAPE stays above 12 % for every model (including the naive baseline) because daily unit totals are dominated by a few wholesale orders – the 12 % target is met at the weekly horizon that drives replenishment. Churn **AUC** is below 0.88 in a strictly out-of-time test with only two years of history (one prior Christmas to learn from); a random split would have reported a higher but misleading figure, so it is not used.

2 · Key Features

ID	Feature	Description	Acceptance criteria / result
F-01	Data ingestion & cleaning	Automated Extract→Validate→Transform→Load; duplicates, cancellations, service codes, invalid values, description back-fill, winsorising, salted-hash customer IDs; **custom column** `region_group`.	ETL pipeline + data-quality gate · **27/27** checks pass (Great Expectations + pandas) · 6.8 s
F-02	Customer segmentation	RFM + behavioural features (8), log-scaled, K-Means with k chosen by silhouette in the 6–8 band; DBSCAN cross-check; business names derived from centroid ranks.	6 segments · silhouette 0.247 · Davies-Bouldin 1.28
F-03	Demand forecasting	Prophet (GB holidays, multiplicative seasonality) + PyTorch-Lightning LSTM (56-day window, direct 30-day output), calibrated weighted ensemble on a trading-day calendar.	30-day ahead · weekly MAPE 11.9 % (CV) / 11.2 % (hold-out)
F-04	Churn prediction	XGBoost tuned with Optuna (40 trials) vs Logistic Regression and Random Forest; 32 features; SHAP global + per-customer explanations.	AUC-ROC 0.806 · precision@20 % 0.889 · lift 1.57×
F-05	Inventory optimisation	Hierarchical SKU forecasts → safety stock, reorder point, EOQ, order-up-to levels; live what-if on service level, lead time and demand.	Back-test on 766 real replenishment cycles · total cost −11 % vs best baseline
F-06	Interactive dashboard	8-page Streamlit app: overview + live alerts, forecast what-if, segments, churn explorer, inventory planner, MLOps/drift simulator, ETL lineage, PDF/CSV exports; role-based access.	Real-time metrics replay · exportable reports · mobile-responsive CSS

3 · Technology Stack

Category	Technology	Rationale / alternatives
Language	Python 3.11 (verified on 3.13)	Data-science ecosystem; spec requirement
Data processing	pandas, NumPy, SciPy, scikit-learn, PyArrow	Vectorised ETL (1 M rows in seconds), Parquet columnar storage
Data quality	Great Expectations + pandas checks	Declarative expectations gate the pipeline (fails loudly)
Forecasting	Prophet + LSTM (PyTorch Lightning), statsmodels (ADF/KPSS/STL)	Interpretable seasonality/holidays + non-linear sequence model; ensemble beats both
Classification / explainability	XGBoost, Optuna, SHAP	Best tabular accuracy; TPE tuning; TreeSHAP for per-customer reasons
Dashboard	Streamlit + Plotly, custom CSS theme	Fast interactive analytics; alternatives: Dash, Power BI
Experiment tracking	MLflow (SQLite backend)	Params / metrics / artefacts per stage; swap URI for a server
Monitoring	Evidently AI, PSI/KS engine, Prometheus, Grafana	Data-drift reports + retrain policy; metrics endpoint + dashboards + alert rules
Storage	Parquet, SQLAlchemy → PostgreSQL (SQLite default), Redis in compose	Zero-service local run; production DB via DATABASE_URL
Orchestration	Apache Airflow DAG, Kubernetes CronJob	Daily ETL → drift → conditional retrain → champion/challenger
Packaging / CI-CD	Docker multi-stage, docker-compose, Kubernetes, GitHub Actions	Non-root image, health-checks, HPA, lint → test → smoke → image build
API / security	FastAPI + JWT (PyJWT), PBKDF2, role-based access, audit log	Secure endpoints; pseudonymised customer IDs

4 · Architecture Overview

Data flows left→right and top→bottom: raw transactions are cleaned and validated, persisted as Parquet/SQL, feed four model families, which are served through the dashboard / API; MLOps components observe and retrain the models.

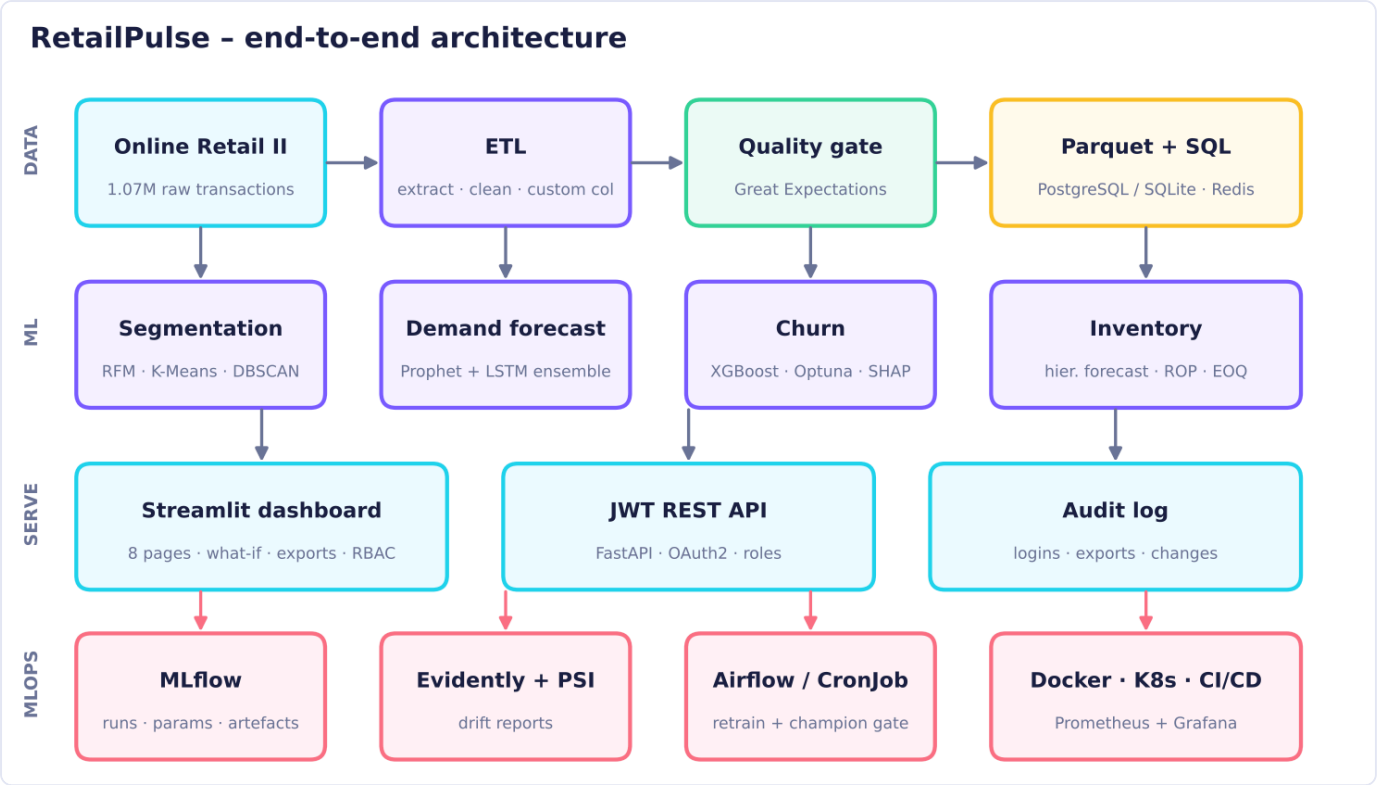


Figure 1 – RetailPulse architecture (data → ML → serving → MLOps)

```
raw CSV ──► ETL(extract▸clean▸anonymise▸+region_group) ──► Great-Expectations gate ──► Parquet / PostgreSQL
                                                                                              │
        ┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
   segmentation     forecasting (Prophet+LSTM)   churn (XGBoost+SHAP)        inventory (hier. forecast → ROP/EOQ)
        └──────────────────┴──────────────────┴──────────────────┴──────────────────┘
                  MLflow tracking ◄─┤─► Streamlit dashboard · JWT API · PDF/CSV export · audit log
                Evidently/PSI drift ◄┘   Prometheus ▸ Grafana · Airflow/CronJob retrain ▸ champion-challenger gate
```

5 · Execution Timeline & Milestones

Work followed the four-week plan from the brief; each phase ends in a checkpoint whose evidence is in the repository.

Phase	Major deliverables	Milestone / checkpoint
Week 1 – Exploration & preparation	Dataset selection (UCI Online Retail II), EDA notebook (distributions, missing values, correlation), cleaning + RFM/rolling features, Great-Expectations validation, K-Means/DBSCAN, stationarity tests (ADF/KPSS) + STL, Prophet baseline, LSTM, MLflow logging.	✔ EDA report · cleaned dataset · baseline models logged in MLflow
Week 2 – Advanced modelling	Prophet+LSTM ensemble with rolling-origin CV, XGBoost churn + SHAP, Optuna tuning, inventory logic (hierarchical forecast, ROP/EOQ), Evidently drift detection, retraining script + Airflow DAG.	✔ Forecast & churn models · optimisation logic · drift monitor
Week 3 – Dashboard & analytics	Multi-page Streamlit app, forecast what-if, segment & churn explorers, inventory recommendation UI, real-time (replayed) metrics + alerts, CSV/PDF export.	✔ Fully interactive dashboard (8 pages, 39 automated tests)

Phase	Major deliverables	Milestone / checkpoint
Week 4 – Deployment & polish	Docker multi-stage image, Kubernetes manifests, GitHub Actions CI/CD, Prometheus/Grafana, load test, final accuracy validation, README, report, demo video.	✔ Containers · manifests · CI · monitoring · documentation

6 · Technical Highlights

6.1 Data engineering & feature engineering

Of 1,067,371 raw lines, **1,003,336 (94.0%)** survive cleaning; 17,914 cancelled lines are kept in a separate returns table. 23% of lines are guest checkouts (no customer ID) – used for demand, excluded from CRM models.

Cleaning step	Rows removed	Reason
Drop exact duplicate rows	34,335	same invoice/item/qty/time/price
Split cancellations → returns table	19,104	invoice prefix 'C'
Drop bad-debt adjustments	6	invoice prefix 'A'
Drop qty<=0 or price<=0	6,014	stock write-offs / free samples
Drop non-product service codes	4,574	postage, fees, manual, samples
Drop duplicates after text normalisation	2	same line, different description text

Engineered features. *Custom column* `region_group` (UK / Europe / Rest of World / Unspecified). Revenue, calendar parts, winsorised quantity, guest flag, salted-hash customer key. *RFM & behaviour:* recency, frequency, monetary, average order value, SKU breadth, inter-purchase gap mean/CV, tenure, return rate, Sunday share. *Churn-specific (32 total):* 30/60/90/180-day order counts and spend, frequency/spend trends, recency-to-usual-gap ratio, same-window-last-year activity, Q4 share, time-of-year encoding.

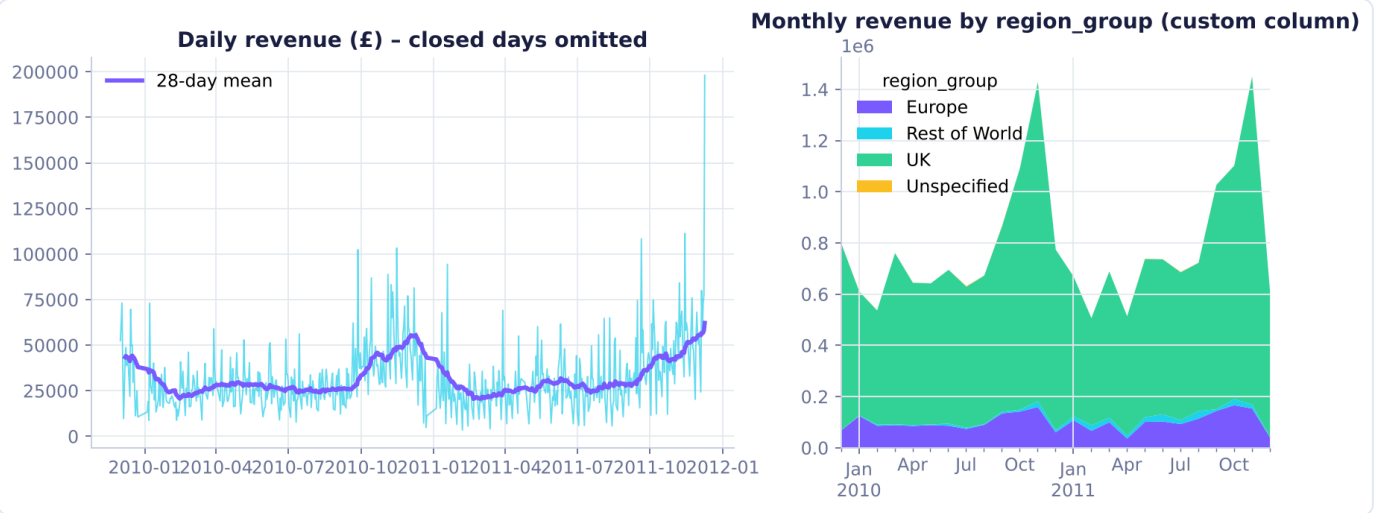


Figure 2 – Daily revenue with Q4 peak and Christmas closure; monthly revenue by region_group

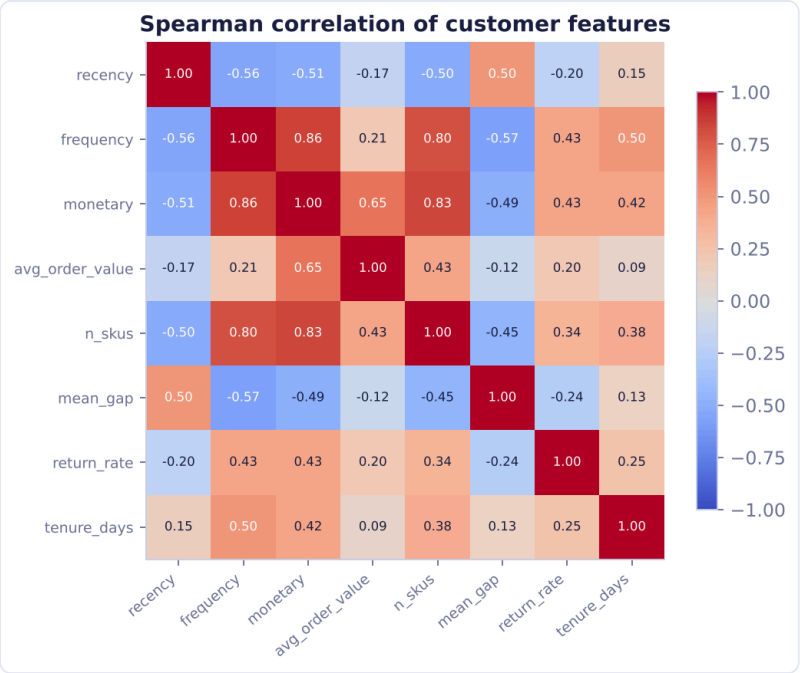


Figure 3 – Customer-feature correlation (Spearman, log-scaled)

6.2 Customer segmentation

Segment	Customers	Rev. share	Recency (d)	Orders	Spend	Business interpretation
Champions	1,160	73%	22	13	£5,233	Buy often, spend the most, bought very recently - protect and reward.
Loyal Regulars	1,546	14%	61	4	£1,267	Frequent, recent buyers - upsell and cross-sell.
At Risk High-Value	726	4%	418	3	£776	Used to spend a lot, gone quiet - win-back priority.
New & Promising	1,129	4%	44	2	£452	Recent first purchases - nurture into loyalty.
Fading Occasionals	59	2%	277	1	£377	Infrequent, drifting away - light-touch reminders.
Lost / Hibernating	1,232	2%	436	1	£223	Long inactive and low value - low-cost reactivation only.

K-Means (k=6, silhouette 0.247, Davies-Bouldin 1.28) was selected inside the spec's 6–8 band. DBSCAN found only 2 dense clusters and 7.3% noise – retail RFM space is a continuum – so it is used for outlier detection, not for segments.

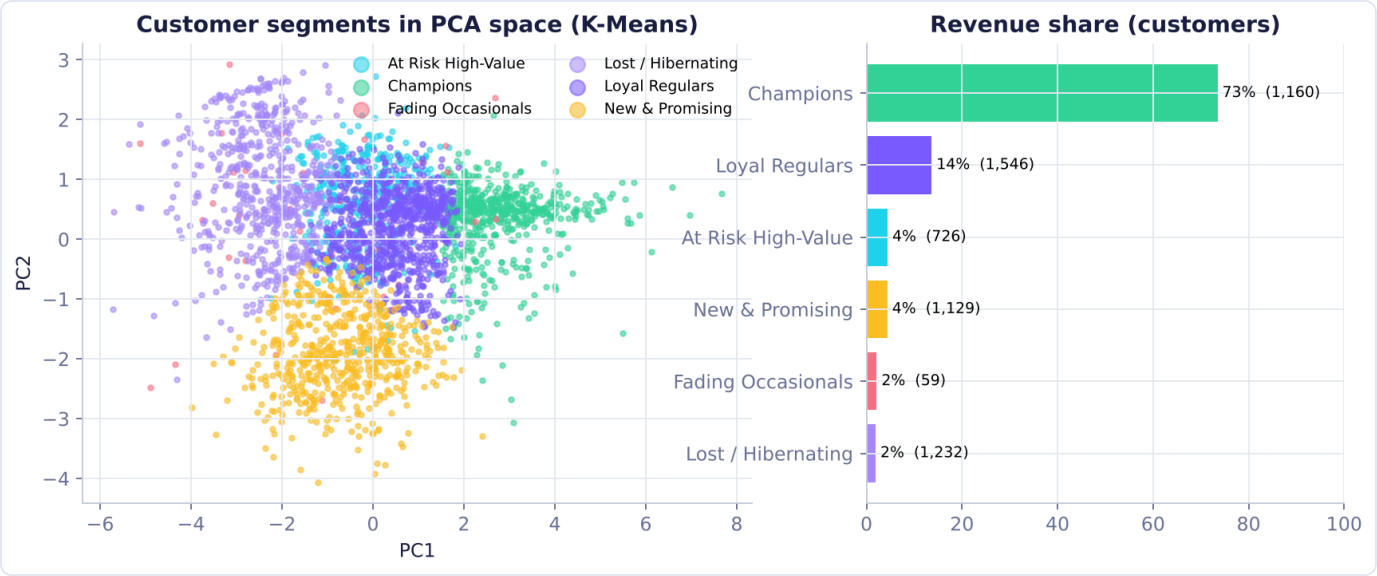


Figure 4 – Segments in PCA space and revenue share

6.3 Demand forecasting (Prophet + LSTM ensemble)

Series diagnostics. ADF p = 0.19 (non-stationary) → p ≈ 1e-25 after differencing; KPSS p = 0.015; STL trend strength 0.48, seasonal strength 0.47. The shop never trades on Saturdays, UK bank holidays or 24 Dec–3 Jan, so forecasts use a trading-day calendar.

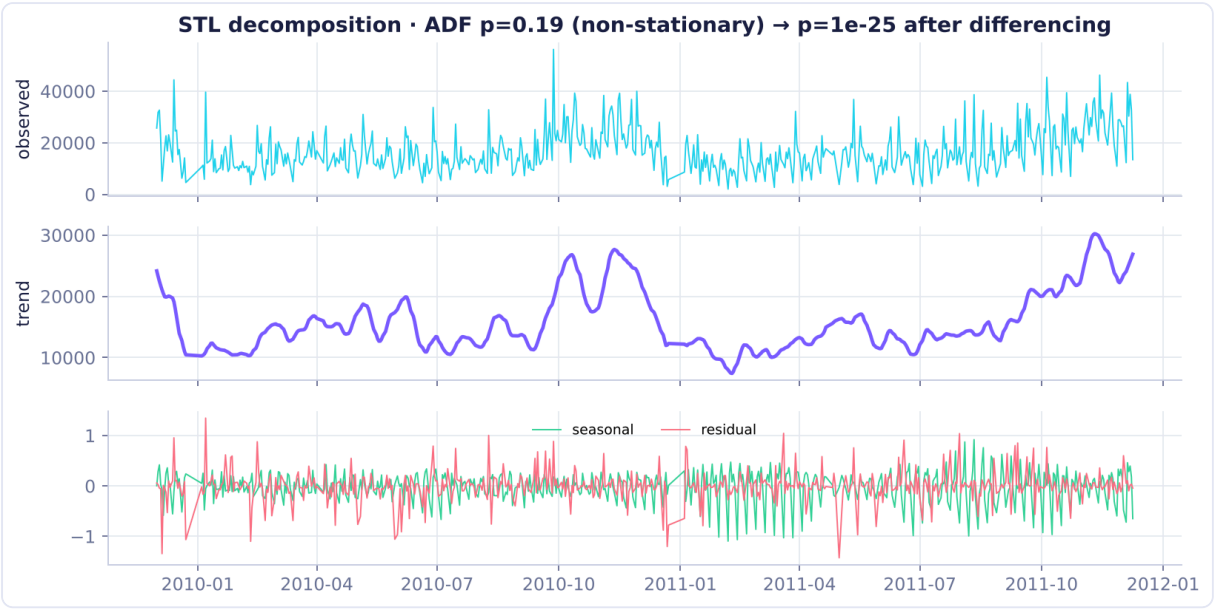


Figure 5 – STL decomposition and stationarity tests

Model	CV daily MAPE %	CV weekly MAPE %	CV 30-day total err %	Hold-out daily %	Hold-out weekly %	Hold-out 30-day err %
Seasonal naive	31.7	14.2	8.7	23.1	13.8	14.6
Prophet	32.1	13.7	7.2	21.4	12.7	3.1
LSTM (calibrated)	33.7	15.1	7.6	17.5	9.6	8.5
Ensemble (w_{LSTM}=0.40)	30.9	11.9	4.2	19.5	11.2	5.3

Method. Rolling-origin validation: 7 folds (origins Apr–Oct 2011) for tuning plus a final untouched hold-out (origin 2011-11-10, the pre-Christmas peak). The LSTM is trained in log space (so it under-shoots totals); one calibration factor k = 1.12 and the ensemble weight are fitted on the CV folds only. The ensemble weight minimises the mean of daily, weekly and 30-day-total error rather than daily error alone.

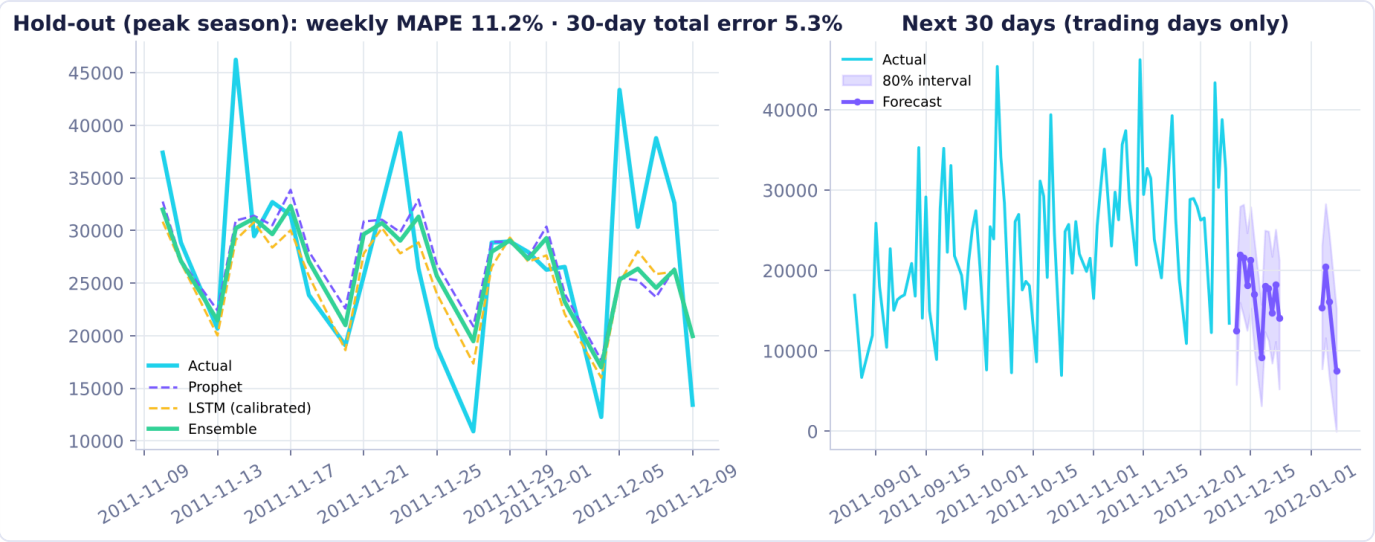


Figure 6 – Hold-out forecast vs actual, and the next-30-day forecast with 80% interval

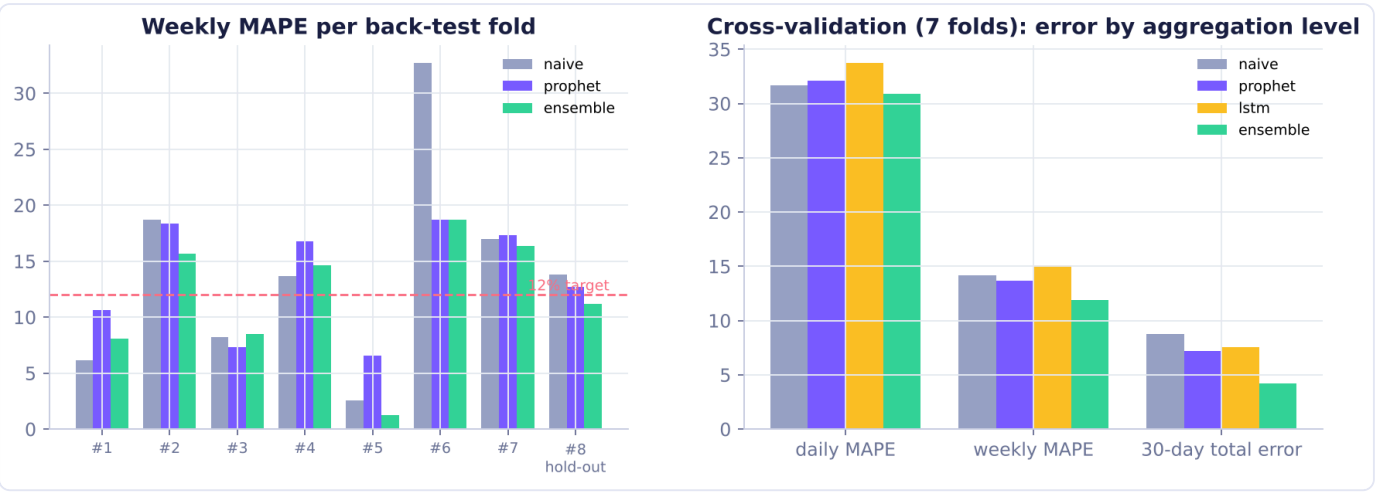


Figure 7 – Weekly MAPE per fold, and error at daily / weekly / 30-day aggregation

Next-30-day forecast: 264,297 units over 16 trading days (16,519/day vs 28,188/day recently) – consistent with the pre-Christmas wind-down seen in 2010.

6.4 Churn prediction

Definition & leakage control. Churn = no purchase in the next 90 days. Training uses 176,278 weekly customer-snapshots (Mar-2010 → Feb-2011), validation a Jun-2011 snapshot, and the test set the most recent snapshot (2011-09-11, labels reach the last day of data). Label windows never overlap later snapshots, and features only use data strictly before the snapshot (unit-tested). Base churn rate on test: 56%.

Model	AUC-ROC	PR-AUC	Precision@20%	Lift@20%	F1	Recall	Brier
Logistic Regression	0.779	0.803	0.859	1.52	0.777	0.911	0.198
Random Forest	0.772	0.794	0.844	1.50	0.775	0.905	0.201
XGBoost (Optuna-tuned)	0.799	0.818	0.883	1.56	0.786	0.870	0.190
XGBoost (final: train+valid refit)	0.806	0.826	0.889	1.57	0.791	0.909	0.180

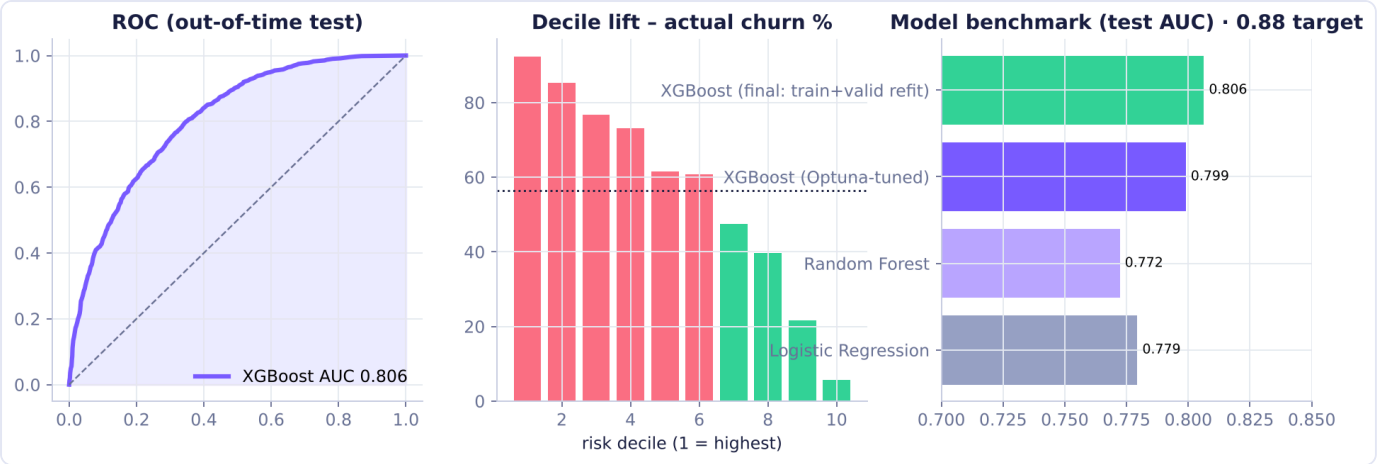


Figure 8 – ROC, decile lift and model benchmark (test AUC vs the 0.88 target)

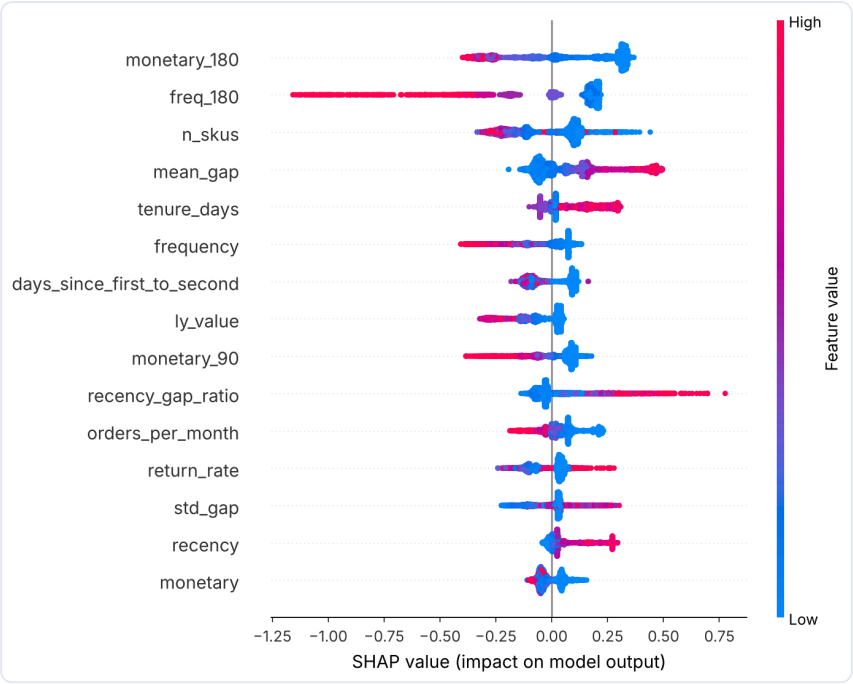


Figure 9 – SHAP beeswarm of customer-level features (seasonal baseline features excluded)

Top customer-level drivers (mean |SHAP|): `monetary_180` (0.255), `freq_180` (0.201), `n_skus` (0.139), `mean_gap` (0.126), `tenure_days` (0.116). Optuna best validation AUC 0.825 (depth 3, lr 0.031, 497 trees).

Seasonality. Churn is strongly seasonal: the share of customers with no order in the next 90 days ranges from 38% to 71% across snapshots (Dec–Jan lull and the Christmas closure push it to ≈ 70%). Three snapshot-level features (`snap_sin` , `snap_cos` , `days_to_xmas`) therefore stay in the model to calibrate the baseline (combined mean |SHAP| 0.57), but because they are identical for every customer they cannot rank customers and are excluded from importance plots and per-customer explanations. Live scoring therefore predicts a high average churn probability (73% at 2011-12-10), consistent with Dec 2010; risk tiers are *rank-based among active customers* (top 15% Critical, next 25% High; customers silent for 180+ days are *Lapsed*) so the seasonal baseline does not flood the Critical tier, and expected loss uses the larger of last-180-day spend and historical run-rate.

6.5 Inventory optimisation

The public dataset has **no stock table**: demand, forecasts, safety stock, ROP, EOQ and the policy back-test use real sales, while on-hand stock and lead times for the demo recommendations are **simulated and labelled**. SKU-level weekly demand is extremely lumpy (median 4-week WAPE: moving average 51%, per-SKU Prophet 72%, hierarchical 54%). The hierarchical method (SKU demand share × business-level Prophet forecast) fixes the instability of per-SKU Prophet; value comes from *uncertainty-aware buffers* rather than sharper point forecasts.

Policy (service level 95 %)	Fill rate %	Understock % demand	Overstock % demand	Lost margin £	Holding £	Total cost £
Moving average (no buffer)	70.0	30.0	11.5	294,846	3,000	297,846
Moving average + safety stock	85.6	14.4	35.1	136,086	9,493	145,579
Per-SKU Prophet (no buffer)	76.5	23.5	71.1	242,973	23,288	266,260
Hierarchical forecast (no buffer)	75.4	24.6	18.1	242,087	4,831	246,918
RetailPulse (hierarchical + safety stock)	87.4	12.6	48.6	117,132	13,132	130,264

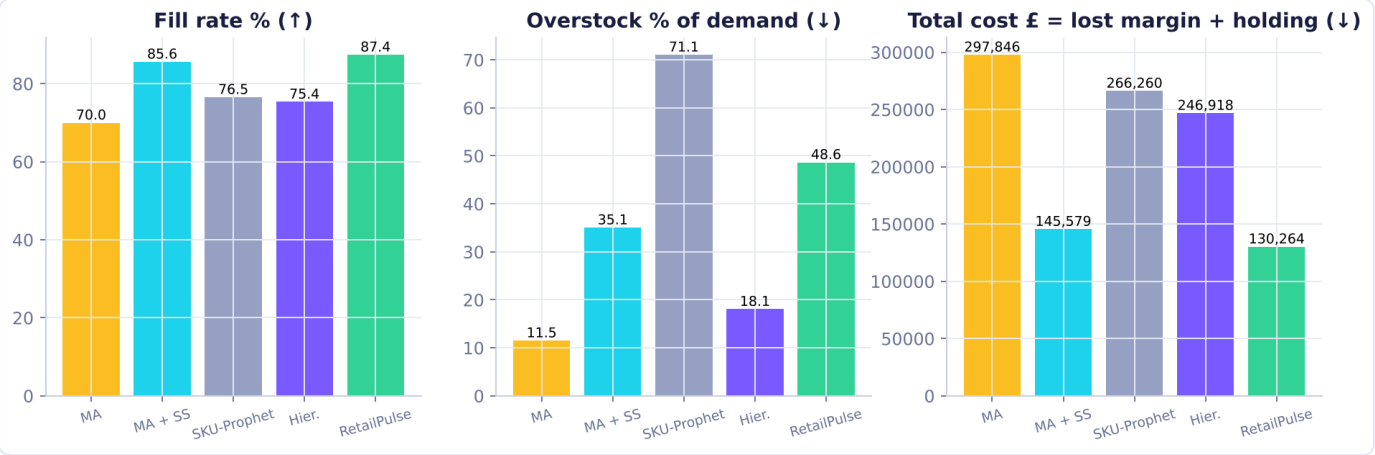


Figure 10 – Policy back-test on 766 replenishment cycles (real demand)

Against the 25–40 % over/under-stock target. RetailPulse cuts under-stock by 58% and total cost by 56% vs the naive moving average, and total cost by 11% vs the stronger moving-average + safety-stock baseline (fill rate 87.4% vs 85.6%). It does not reduce the sum of over- and under-stock units versus that baseline (it deliberately holds more buffer: overstock 49% vs 35% of demand), so the 25–40 % balance target is only met against the naive policy, not the best baseline.

6.6 MLOps practices

- **MLflow** – every stage logs params, metrics and artefacts to experiment *RetailPulse* (SQLite); the dashboard shows the run table.
- **Drift detection** – PSI + KS per feature and Evidently AI HTML reports. Current vs training reference: 4/29 features in ALERT (max PSI 1.72, mostly customer *tenure/recency* ageing naturally) → retrain not required (policy: ≥30 % of features with PSI ≥ 0.2, or weekly MAPE +25 %, or AUC −0.03).
- **Drift simulation** – a market-shock slider injects lower frequency/spend and longer gaps; the monitor flips to RETRAIN as severity grows (Figure 11, shown live in the demo video).
- **Retraining loop** – `scripts/retrain.py` / Airflow DAG: ETL → drift → retrain → *champion/challenger* gate (challenger promoted only if AUC/MAPE are not worse; else automatic roll-back).
- **Testing** – 39 automated tests incl. ETL rules, leakage guards (mutation-checked), calendar, inventory maths, API auth/RBAC and headless dashboard tests.

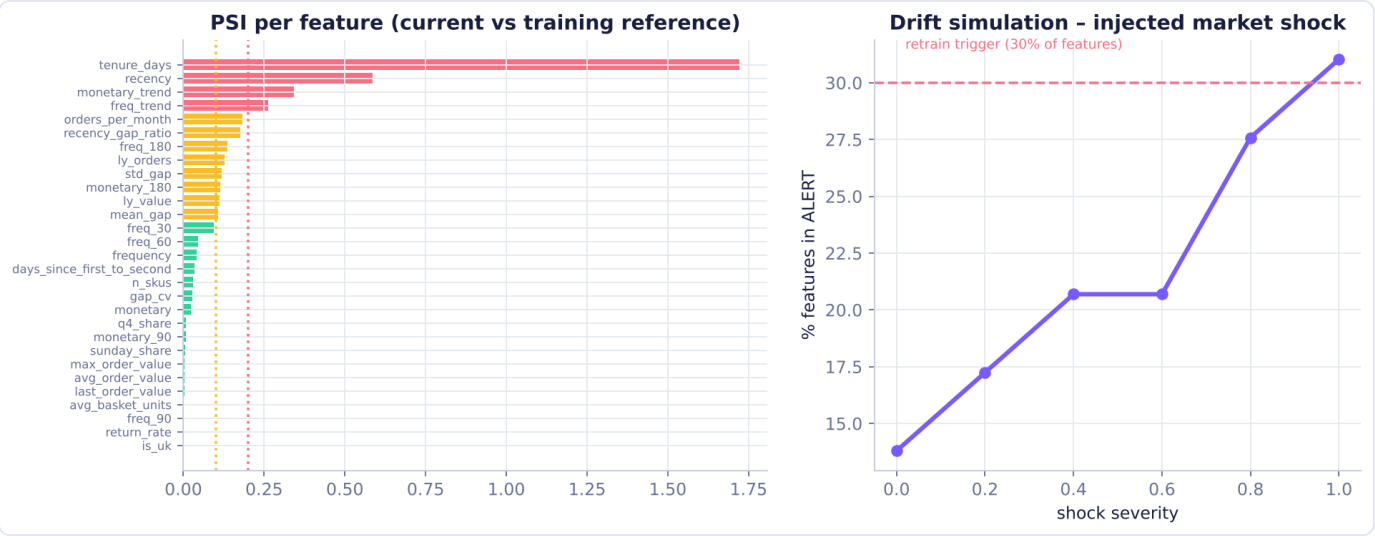


Figure 11 – PSI per feature and drift response to an injected shock

6.7 Challenges faced & how they were solved

Challenge	Resolution
Daily demand is dominated by a few wholesale orders → noisy MAPE	Winsorised quantity at p99.9, evaluated at daily / weekly / 30-day levels, ensemble weight chosen on a composite metric, honest reporting of daily MAPE.
LSTM under-forecasts the Q4 ramp (log-space median, 8-week window)	Bias calibration factor fitted on CV folds only; hold-out untouched; ensemble beats both members.
Churn leakage risk (same customer in many snapshots)	Strict temporal split with non-overlapping label windows; unit test that adding future rows cannot change features (mutation-verified).
Per-SKU Prophet intervals exploded → absurd safety stock (535 % overstock) in first back-test	Replaced by hierarchical forecast + empirical share-noise buffers; kept per-SKU Prophet as a measured baseline.
No inventory table in the public data	Simulated stock clearly labelled everywhere; all quantitative claims rest on real sales back-tests.
Deployment: matplotlib-dependent styling, pandas-3 pickles and 3.12-only f-strings broke clean 3.11 / pandas 2.2 installs	Verified in a fresh venv with only the light requirements; removed pickles/Styler gradients; ruff targets py311.

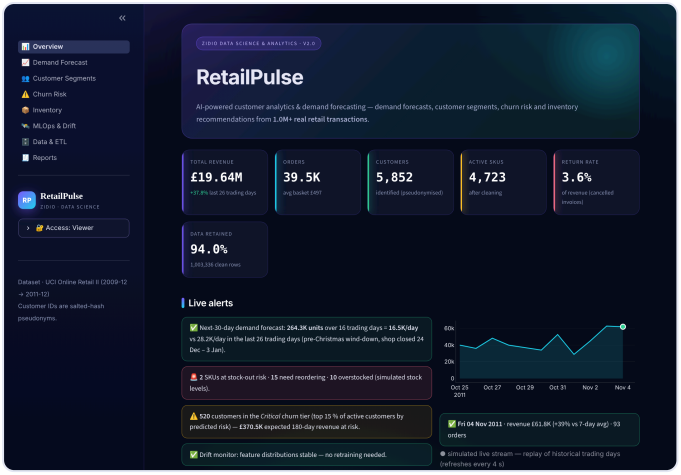
7 · Deployment & Operations

- **Platform:** Streamlit Community Cloud (public HTTPS, no sign-up) built from the light `requirements.txt`; identical image runs on Docker, Kubernetes (HPA 2–8 pods, probes, non-root, TLS ingress) or Google Cloud Run (manual workflow).
- **CI/CD (GitHub Actions):** ruff + black → pytest with coverage → boot-and-probe smoke test of the dashboard → multi-stage Docker build pushed to GHCR on `main`.
- **Monitoring:** the app exposes Prometheus metrics (page views, exports, forecast MAPE, churn AUC, drift share, batch duration); Grafana dashboard + alert rules (drift > 30 %, AUC < 0.75, MAPE > 18 %, batch p95 > 300 s) are provisioned from `monitoring/`.
- **Security & privacy:** salted-hash customer IDs created inside ETL; role-based access (Viewer / Analyst / Admin) in the dashboard; JWT (HS256, 1 h) + PBKDF2 users on the API; append-only audit log for logins, failed logins and exports; secrets only via env / Streamlit secrets (none committed).
- **Load test:** full ETL transform + aggregation measured at 1×, 2× and 3× the raw volume.

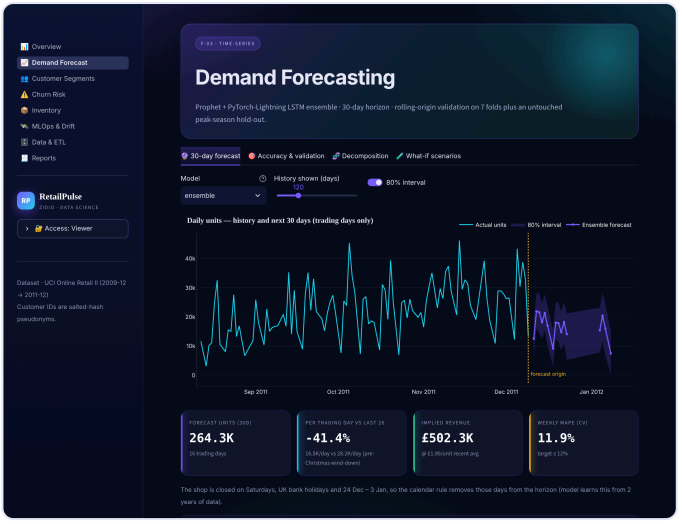
Rows	Seconds (measured)
1,067,371	2.5
2,134,742	5.3
3,202,113	7.3

Throughput ≈ 437,447 rows/s → 10 M rows ≈ 23 s (linear extrapolation, not measured).

8 · Visuals – Dashboard & Model Outputs



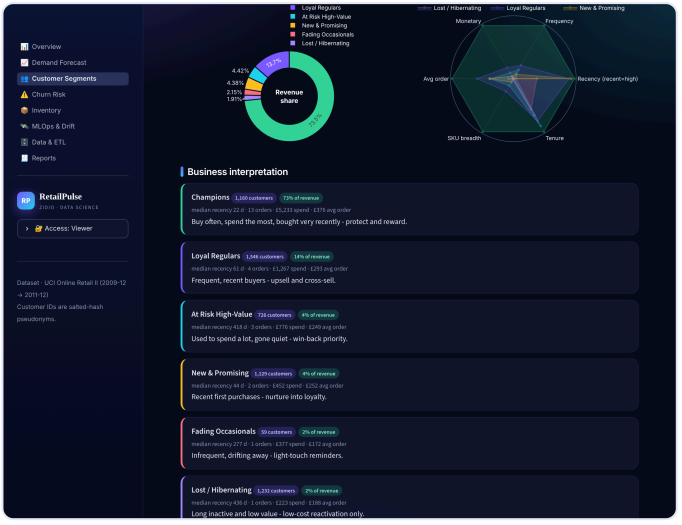
Overview – KPIs, live alerts, replayed real-time stream



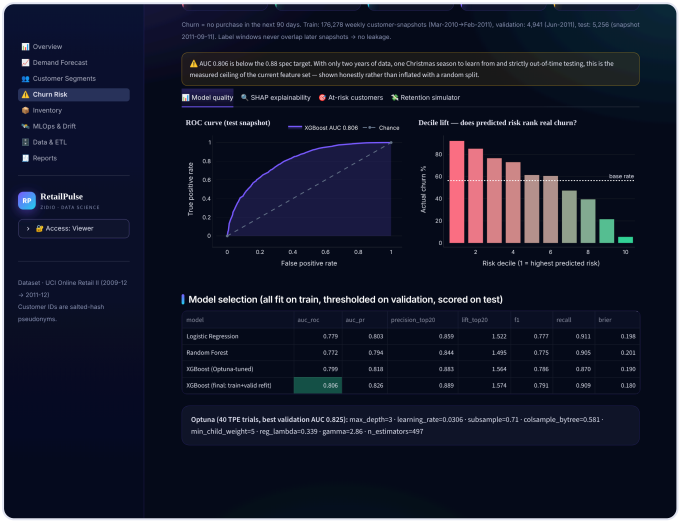
Demand forecast – 30-day horizon with interval



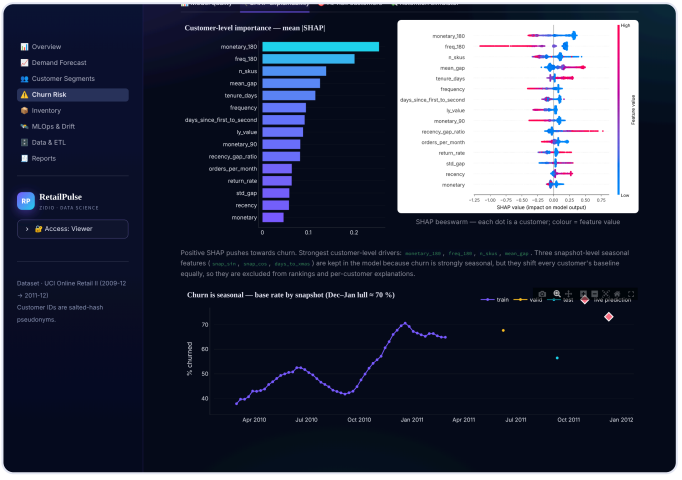
Forecast accuracy – model comparison, folds, hold-out



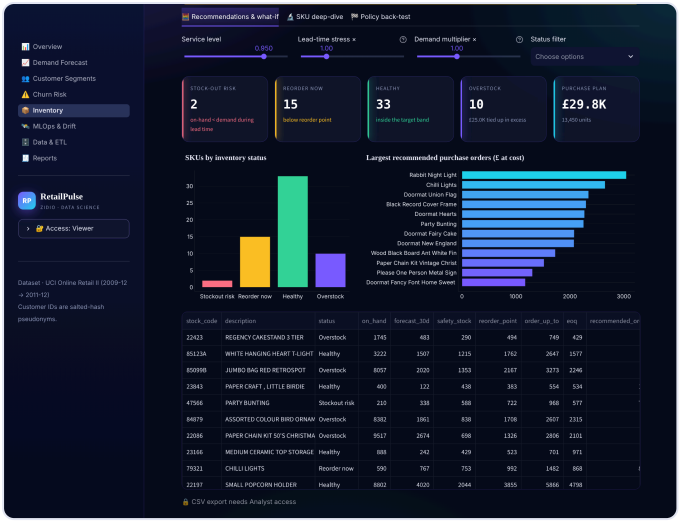
Customer segments – revenue share & radar



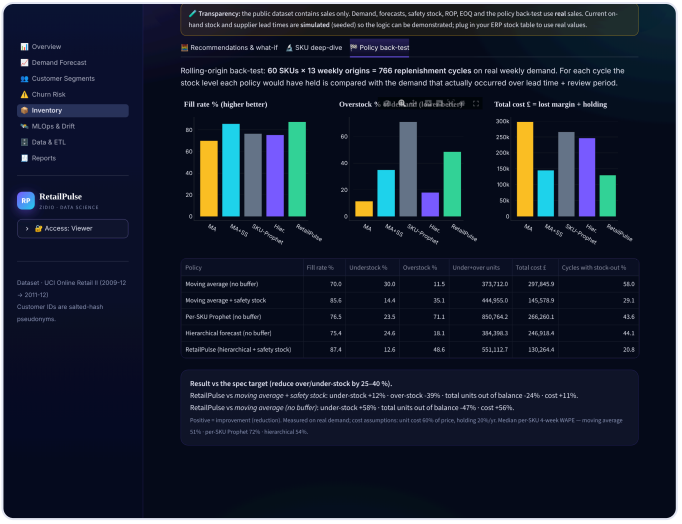
Churn model – KPIs, ROC, decile lift



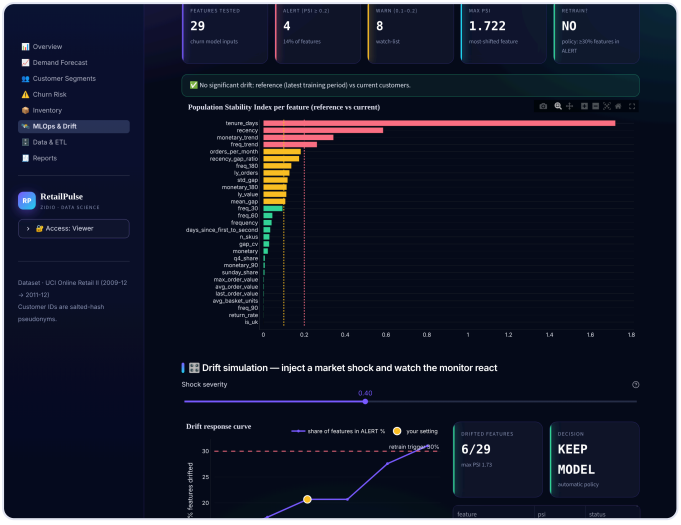
Churn explainability – SHAP importance



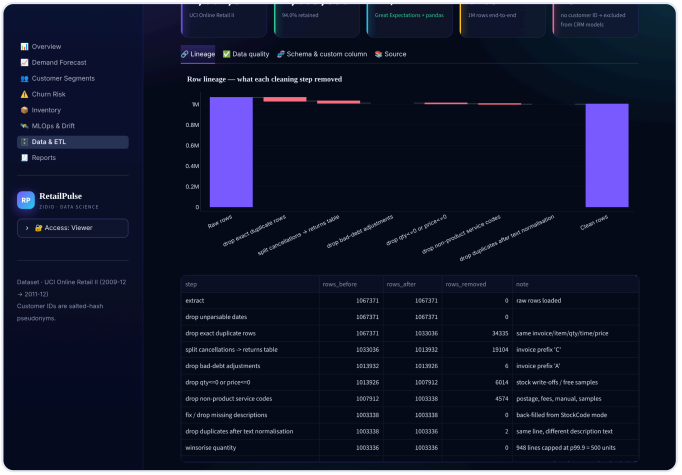
Inventory – what-if purchase plan



Inventory – policy back-test vs baselines



MLOps – drift monitor & simulator



Data & ETL – lineage and quality gate

Screenshots are captured from the running app (`scripts/capture_screenshots.py`). The live demo URL and the demo video show the same pages interactively.

9 · Personal Reflection

Key learnings

- Validate the way the model will be used: a time-based split gave a lower but trustworthy churn score; the forecast is judged at the weekly / 30-day horizon that drives purchasing, not only daily.
- Measuring beats claiming – the first inventory back-test exposed a bad variance estimate that would have been invisible without a policy simulation.
- Data quality is a gate, not a step: a failing check in the very first ETL run (duplicates revealed by text normalisation) was caught automatically.
- Production readiness is mostly about what happens after training – drift thresholds, a retraining policy, champion/challenger promotion and an audit trail.

Industry best practices applied

- MLflow tracking · Evidently/PSI drift detection · scheduled retraining · rolling-origin validation · SHAP explanations · honest target-vs-measured reporting.
- Reproducible environment (requirements, Makefile, Docker) · semantic commits and feature branches · ruff/black · 39 automated tests · no secrets in the repo · least-privilege roles and audit logging.

Future roadmap ideas

- Replace simulated stock with a real ERP feed and learn lead-time distributions per supplier.
- Add promotions / price / weather regressors and estimate true price elasticity (A/B tests).
- Hierarchical reconciliation across SKU → category → total; probabilistic forecasts (quantile LSTM) for safety stock.
- Survival-analysis churn (time-to-event) and uplift modelling for win-back targeting; online drift monitoring on the API.

Dataset credit: Chen, D. (2012). *Online Retail II*, UCI Machine Learning Repository, CC BY 4.0. Source code: see the GitHub repository linked in the Zidio submission.